NEONECY TEAM APP — Backend Issues, Contracts & UI Implementation Guide

Date: 17 October 2026 | **Host:** https://api.bingi.com.bd/api | **Focus:** Issues, Missing Endpoints & Full Specifications

This document compiles all live backend exceptions, security vulnerabilities, schema discrepancies, and missing API contracts identified during mobile integration testing across Chat, Push Notifications, Attendance, Employee Onboarding, Leads, Tasks, Field Visits, Corporate Expenses, Audit Logs, Scheduled SMS Reminders, and Certificates & Testimonials Management. High-resolution mobile screenshots and exact UI-to-backend data flows are included to guide complete, flawless backend implementation.

IN-APP CHAT, MESSAGING & PUSH NOTIFICATIONS

1. Employee Messages to Administrator Trigger Zero In-App Notifications and Zero Push Notifications

```
POST /team-operations/messages/{conversation_id}/send (employee token)
POST /team-operations/messages (employee token)
GET /team-operations/notifications (administrator token)
```

CRITICAL NOTIFICATION FLAW: When an employee sends a direct or team message to an administrator, the server returns HTTP 201 'Message sent.' but COMPLETELY FAILS to generate any in-app notification record or dispatch any FCM push notification to the administrator.

Live Production Verification:
1. Super Admin (user_id=1, dragon@bingi.com.bd) registered an active Android FCM push device token (device_id=16) via POST /push-devices.
2. Employee Yeapas (user_id=66, employee_id=2) sent a message to the administrator via POST /messages/1/send. Server responded HTTP 201.
3. Querying GET /notifications for Super Admin returned ZERO new notification entries (unread_count remained unchanged at 107; no notification row inserted).
4. Querying GET /push-deliveries confirmed that NO FCM push delivery was ever queued, recorded, or dispatched to user_id=1.
5. In reverse: When Super Admin sent a message to the employee in the exact same conversation, the server successfully created in-app notification id 25879 ('New message from Super Admin') and triggered push delivery.

Root Cause Analysis:
• Super Admin has user_id=1, role=SUPER_ADMIN, but has NO record in the 'employees' database table (employee.id is 0 or null). POST /messages rejects Admin recipients with HTTP 422 ('The selected recipient employee id is invalid.' / 'Select a staff recipient.').
• When a message is sent, the backend notification dispatcher resolves recipients strictly by looking up conversation_participants.employee_id and mapping to employees.user_id. Because the administrator is not an employee row in the participants table, the listener completely skips user_id=1.

Request Body / Query:

```
{
  "conversation_id": 1,
  "sender_employee_id": 2,
  "recipient": "Super Admin (user_id: 1)",
  "body": "Hello Admin direct test"
}
```

Observed Response / Evidence:

```
{
  "message_send_status": 201,
  "item": {
    "id": 56,
    "body": "Hello Admin direct test",
    "sender": {
      "user_id": 66,
```

```
      "employee_id": 2,
      "name": "Yeapas"
    }
  },
  "admin_in_app_notifications_created": 0,
  "admin_push_deliveries_attempted": 0,
  "verified_reverse_direction": {
    "admin_to_employee_notification_created": true,
    "created_notification_id": 25879,
    "title": "New message from Super Admin"
  }
}
```


Required Fix / Expected Contract:

1. Update POST /team-operations/messages to accept direct conversation creation with administrators via recipient_user_id (e.g. 1 / 'usr_1').
2. In the message sending controller / event listener (e.g. ConversationMessageController@send), resolve conversation participants by user_id in addition to employee_id.
3. When any employee sends a message in a conversation involving an administrator (or team chat supervised by admin), create an in-app notification record in the 'notifications' table for user_id=1 with type='message', entity.type='team_conversation', and data.event='message_received'.
4. Dispatch an FCM push notification payload to all registered device tokens for user_id=1 via Firebase Cloud Messaging.

2. Admin message reactions fail with HTTP 422 (Missing Employee Profile)

```
POST /team-operations/messages/items/{message_id}/reactions (Super Admin token)
```

When Super Admin (user_id=1, dragon@bingi.com.bd) attempts to react to an employee's message with an emoji, the request fails with HTTP 422. The backend responds with "An employee profile is required to react.". The administrator cannot react to or acknowledge team updates without an artificial separate employee profile.

Request Body / Query:


```
{
  "emoji": "\ud83d\udc4d"
}
```


Observed Response / Evidence:


```
{
  "http_status": 422,
  "message": "An employee profile is required to react."
}
```


Required Fix / Expected Contract:

Allow user_id=1 / Administrator to add and toggle reactions on messages. Automatically map user_id=1 to the primary administrator employee profile (e.g. emp_mahmud / Super Admin).

3. Uploading a group chat photo sends noisy message notification to all members

```
POST /team-operations/messages/{conversation_id}/send (with body '[Group Avatar]' + photo)
```

When an administrator updates a group chat's photo/avatar by sending an image attachment, the server treats it as a standard message and generates push notifications and feed cards with body '[Group Avatar]' to all group members. Members receive unwanted alerts for a silent configuration change.

Request Body / Query:


```
{
  "body": "[Group Avatar]",
  "attachment": "<binary image/png>"
}
```


Observed Response / Evidence:


```
{
  "notification_created": true,
  "title": "New message from Super Admin",
  "body": "[Group Avatar]"
}
```

Required Fix / Expected Contract:
Provide a dedicated group photo endpoint (e.g. POST /team-operations/messages/{id}/avatar or PATCH /team-operations/messages/{id}), or suppress push notifications when the message body indicates an internal metadata update event.

4. No server endpoints to add members or leave a group chat (HTTP 404)

```
POST /team-operations/messages/{conversation_id}/add-member
```

```
POST /team-operations/messages/{conversation_id}/leave
```

The backend returns HTTP 404 for member addition and group departure. Employees cannot leave team/department groups they were placed into, and administrators have no documented API to dynamically add members to existing groups.

Request Body / Query:

```
{
  "employee_id": 3
}
```

Observed Response / Evidence:

```
{
  "http_status": 404,
  "message": "Route not found."
}
```

Required Fix / Expected Contract:
Document and implement authorized membership management contracts: POST /messages/{id}/add-member (admin only) and POST /messages/{id}/leave (current employee).

5. Employees cannot archive conversations (HTTP 403 Forbidden)

```
POST /team-operations/messages/{conversation_id}/archive (employee token)
```

When an employee attempts to archive a direct or group conversation in their inbox, the backend rejects the request with HTTP 403: "Your role does not allow group management.". Archiving is an individual user inbox preference, not a global group administration action.

Request Body / Query:

```
{}
```

Observed Response / Evidence:

```
{
  "http_status": 403,
  "message": "Your role does not allow group management."
}
```

Required Fix / Expected Contract:
Make conversation archiving user-scoped so that any participant can archive/hide a conversation from their own personal inbox view without requiring group administration permissions.

6. Duplicate notification entries created in employee feed

```
GET /team-operations/notifications?filter=all&per_page=100
```

A single sent message or single work item assignment frequently creates multiple identical notification records in the database with identical titles, bodies, and creation timestamps.

Observed Response / Evidence:

```
{
  "duplicate_sample": {
    "title": "New message from Super Admin",
    "occurrences": 2,
    "time": "2026-10-15T13:15:24+06:00"
  }
}
```

Required Fix / Expected Contract:
Enforce idempotency on server-side notification generation; ensure only one notification record is created per event and recipient.

7. Follow-Up Due Reminders and Overdue Alerts Are Dispatched ONLY to Admin (Assigned Employee Receives Zero Alerts)

```
Background Scheduled Cron / Worker (followups.reminder & followups.overdue)
GET /team-operations/notifications (Employee token)
GET /team-operations/push-deliveries
```

CRITICAL WORKFLOW FLAW: When a follow-up is scheduled with a reminder (e.g. 15 minutes before due time) or when a follow-up becomes overdue, the backend background reminder daemon triggers notification cards and FCM push notifications ONLY to Super Admin (user_id=1). The employee to whom the follow-up is assigned receives ZERO in-app notifications and ZERO push notifications.

Live Production Verification & Evidence:
1. Follow-up #25 ('Sea Pearl Beach Resort & Spa') was created with assignee_employee_id=2 (Yeapas / user_id=66), scheduled for 5:44 PM.
2. When the 15-minute reminder fired (17:28:00), the server created in-app notification #14398 ('Follow-up due in 15 minutes') and push delivery #122 exclusively for user_id=1 (Super Admin).
3. When the follow-up became overdue (17:45:01), the server created notification #14470 ('Followup overdue') and push delivery #123 exclusively for user_id=1 (Super Admin).
4. Querying the employee's inbox (GET /notifications) confirmed that the employee received the initial creation card (#14006), but received ZERO reminder cards and ZERO overdue cards.
5. Querying GET /push-deliveries confirmed that 100% of all 87 push dispatches across the entire backend database were targeted to user_id=1. Zero push dispatches exist for any assigned staff employee.

Request Body / Query:

```
{
  "follow_up_id": 25,
  "lead_name": "Sea Pearl Beach Resort & Spa",
  "scheduled_at": "2026-10-15T17:44:00+06:00",
  "assignee_employee_id": 2,
  "reminder_minutes": 15
}
```

Observed Response / Evidence:

```
{
  "admin_inbox_notifications": [
    {
      "id": 14398,
      "title": "Follow-up due in 15 minutes",
      "recipient_user_id": 1,
      "time": "2026-10-15T17:28:00+06:00"
    },
    {
      "id": 14470,
      "title": "Followup overdue",
      "recipient_user_id": 1,
      "time": "2026-10-15T17:45:01+06:00"
    }
  ],
  "employee_inbox_notifications": [
    {
      "status": "none_received",
      "due_reminder": false,
      "overdue_alert": false
    }
  ],
  "push_deliveries_table_analysis": {
    "total_deliveries": 87,
    "deliveries_to_admin_user_1": 87,
    "deliveries_to_assigned_staff": 0
  }
}
```

Required Fix / Expected Contract:
In the background scheduler / cron worker that checks scheduled follow-ups (and tasks/visits):
1. Resolve the assigned employee (followup.assignee_employee_id) and map to their linked user account (e.g. User::where('employee_id', $followup->assignee_employee_id)->first()).
2. Insert the 'followups.reminder' and 'followups.overdue' notification rows into the 'notifications' table for the ASSIGNED EMPLOYEE's user_id so it appears in their in-app notification inbox.
3. Dispatch the FCM push notification to all active device tokens registered by the ASSIGNED EMPLOYEE in 'push_devices'.
4. While alerting administrators is useful as a supervisory rollup, the primary assignee MUST receive both the in-app notification and FCM push alert.

SECURITY, PERMISSIONS & WORKSPACE

8. Employee department roster exposes coworker payroll and banking

```
GET /team-operations/my-department (ordinary employee token)
```

A standard TEAM_OPERATIONS employee with assigned scope received other employees' complete salary objects via /my-department. The roster includes exact salary amounts, bank name, account number, branch, and SWIFT/routing numbers. Sensitive values are redacted below.

Observed Response / Evidence:

```
{
  "salary": {
    "base_salary": "<redacted>",
    "bank_account": "<redacted>"
  }
}
```

Required Fix / Expected Contract:
Use a public colleague serializer for department/team rosters; expose only approved directory fields (name, designation, avatar, phone, email). Completely strip salary, payroll, and banking details.

9. Employee offboarding summary fails with HTTP 500

```
GET /team-operations/team/employees/2/offboarding-summary (admin)
```

```
GET /team-operations/team/employees/8/offboarding-summary (admin)
```

The newly advertised employee offboarding summary endpoint returns HTTP 500 for both existing employees and freshly created QA employees. Administrators cannot inspect pending leads, tasks, follow-ups, or reports before suspending or offboarding.

Observed Response / Evidence:

```
{
  "http_status": 500,
  "message": "Server Error"
}
```

Required Fix / Expected Contract:
Return the documented offboarding summary structure with counts and item arrays of active leads, tasks, follow-ups, visits, and direct reports for reassignment.

10. Team update fails for an existing team with HTTP 500

```
PATCH /team-operations/team/teams/{id} (admin)
```

A team was created successfully with ID 8. Sending a PATCH request to update its description and lead returned HTTP 500, even though GET detail and DELETE work for that same team.

Request Body / Query:

```
{
  "description": "QA update verified",
  "lead_id": 9
}
```

Observed Response / Evidence:

```
{
  "http_status": 500,
  "message": "Server Error"
}
```

Required Fix / Expected Contract:
Accept authorized team updates (name, description, lead_id) and return the updated team record with HTTP 200.

11. Role catalog, role detail and role update fail with HTTP 500

```
GET /team-operations/administration/roles (admin)
```

```
GET /team-operations/administration/roles/{id} (admin)
```

```
PATCH /team-operations/administration/roles/{id} (admin)
```

The role catalog returns HTTP 500. A new custom QA role was created successfully with ID 30, but GET and PATCH on that existing role return HTTP 500. This prevents administrators from managing role permissions.

Request Body / Query:

```
{
  "description": "QA updated",
  "permissions": [
    "team.leads.view"
  ]
}
```

Observed Response / Evidence:

```
{
  "http_status": 500,
  "message": "Server Error"
}
```

Required Fix / Expected Contract:
Return the documented successful response for authorized valid requests, with validation/permission errors reported as 4xx instead of an unhandled 500.

12. Administration overview summary fails with HTTP 500

```
GET /team-operations/administration (admin)
```

The administrator overview dashboard returns HTTP 500, preventing aggregated administrative counts and health metrics.

Observed Response / Evidence:

```
{
  "http_status": 500,
  "message": "Server Error"
}
```

Required Fix / Expected Contract:
Return aggregated metrics or a structured 4xx error with actionable validation details.

ATTENDANCE TRACKING & WORKDAY

13. Ending an attendance break fails with HTTP 500

```
POST /team-operations/attendance/break/end (employee token)
```

After successful check-in and break start on an active employee, calling break/end returns HTTP 500. The workday remains stuck in 'on_break' state and the employee cannot resume work.

Request Body / Query:

```
{}
```

Observed Response / Evidence:

```
{
  "http_status": 500,
  "message": "Server Error"
}
```

Required Fix / Expected Contract:
Return the updated attendance state with status 'checked_in' and close the active break. Inspect the server production exception log for this request.

14. Attendance check-out fails with HTTP 500

```
POST /team-operations/attendance/check-out (employee token)
```

Submitting check-out details with location metadata (GPS latitude/longitude, accuracy, device ID, Wi-Fi SSID, checkout summary, and work notes) crashes with HTTP 500.

Request Body / Query:

```
{
  "latitude": 23.8103,
  "longitude": 90.4125,
  "accuracy": 12.4,
  "device_id": "qa-device-id",
  "wifi_ssid": "QA Office Wi-Fi",
  "summary": "Completed day tasks",
  "work_notes": "All follow-ups processed."
}
```

Observed Response / Evidence:

```
{
  "http_status": 500,
  "message": "Server Error"
}
```

Required Fix / Expected Contract:
Successfully record the check-out timestamp, calculate worked minutes, persist location/device metadata, and return the completed attendance record.

15. Attendance returns negative break duration (-360 minutes)

```
GET /team-operations/attendance/today
```

```
GET /team-operations/attendance/admin/employees/8?date=2026-10-15
```

Seconds after starting a break, the API response reported approximately minus 360 break minutes (-359.96). This is caused by timezone arithmetic mismatch between UTC server storage and Asia/Dhaka local time (+06:00).

Observed Response / Evidence:

```
{
  "id": 2,
  "date": "2026-10-15",
  "status": "on_break",
  "check_in_at": "2026-10-15 12:41:21+00",
  "break_started_at": "2026-10-15 12:41:22+00",
  "break_minutes": -359.96237813333335,
  "worked_minutes": 0.01666666666665151
}
```

Required Fix / Expected Contract:
Use consistent timestamp arithmetic in UTC or Asia/Dhaka. Break and worked minutes must be non-negative elapsed durations.

16. Attendance metadata (GPS, Wi-Fi, summary, notes) has no read-back fields

```
POST /team-operations/attendance/check-in
```

```
GET /team-operations/attendance/admin/employees/{employee_id}
```

Location-aware check-in accepts GPS coordinates, accuracy, device ID and Wi-Fi SSID. However, subsequent employee and admin attendance reads return timestamps and durations only. Persisted location and device metadata cannot be verified or

displayed in administrative dashboards.

Request Body / Query:

```
{
  "latitude": 23.8103,
  "longitude": 90.4125,
  "accuracy": 12.4,
  "device_id": "qa-script-device",
  "wifi_ssid": "QA verification network"
}
```

Observed Response / Evidence:

```
{
  "attendance_response_fields": [
    "id",
    "date",
    "status",
    "check_in_at",
    "check_out_at",
    "break_started_at",
    "break_minutes",
    "worked_minutes",
    "target_minutes",
    "is_late"
  ]
}
```

Required Fix / Expected Contract:
Include authorized location, device, Wi-Fi SSID, checkout summary, and work notes in the attendance detail response for administrative verification.

17. No documented attendance-request cancellation or overtime-revocation API

```
Missing: employee cancellation of an attendance request
```

```
Missing: administrator revocation of approved overtime
```

The UI supports withdrawing requests and revoking overtime, but the Postman collection documents only request creation and administrator approval. No cancellation or revocation endpoint is specified.

Request Body / Query:

```
{
  "example_request_id": 9,
  "action": "cancel",
  "reason": "Employee withdrew the request"
}
```

Required Fix / Expected Contract:
Provide the actual route, method, payload and allowed status transitions for attendance request cancellation and overtime revocation.

18. No documented attendance holiday edit or delete API

```
Missing: update/delete /team-operations/attendance/admin/holidays/{holiday_id}
```

The collection includes POST to create a holiday and GET settings to read holidays, but no server mutation contract exists to update or delete an existing holiday.

Request Body / Query:

```
{
  "holiday_id": "<existing id>",
  "action": "delete"
}
```

Required Fix / Expected Contract:
Document and implement authorized holiday edit and delete operations (e.g. PUT/DELETE /team-operations/attendance/admin/holidays/{id}).

19. Admin overview counts inactive/offboarded employees in attended_today, conflicting with staff list

```
GET /team-operations/attendance/admin/overview?date=2026-10-15 (admin)
```

```
GET /team-operations/attendance/admin/staff?date=2026-10-15 (admin)
```

GET /attendance/admin/overview aggregates attendance across all employee records in the database (returning attended_today: 2, absent: 8, total 10 staff), counting attendances belonging to inactive/offboarded test employees (IDs 8 & 10). In contrast, GET /attendance/admin/staff strictly filters by active employees only (7 staff), returning 0 check-ins (all 7 are not_checked_in). This causes a direct data inconsistency where the admin home card displays '2 / 7 Staff checked in', but the Attendance Admin detail screen displays '0 Attended today' with all staff marked 'Not checked in'.

Observed Response / Evidence:


```
{
  "overview_metrics": {
    "working_now": 1,
    "attended_today": 2,
    "late_arrivals": 1,
    "absent": 8,
    "total_counted_staff": 10
  },
  "staff_list_summary": {
    "total_staff_returned": 7,
    "all_employment_status": "active",
    "attended_count": 0,
    "status_of_all_7": "not_checked_in"
  },
  "root_cause_records": {
    "employee_8": {
      "name": "QA Oct15 1015124110 1",
      "employment_status": "inactive",
      "attendance": {
        "status": "on_break",
        "check_in_at": "2026-10-15 12:41:21+00"
      }
    },
    "employee_10": {
      "name": "QA Oct15 contract 1015130203",
      "employment_status": "inactive",
      "attendance": {
        "status": "checked_out",
        "check_in_at": "2026-10-15 00:00:00+00"
      }
    }
  }
}
```


Required Fix / Expected Contract:

Align the SQL aggregation in GET /attendance/admin/overview to filter by active employees only (WHERE employees.employment_status = 'active') so that overview metrics match the staff list returned by GET /attendance/admin/staff.

LEADS & POSTMAN COLLECTION

20. Mobile permitted-leads list fails with HTTP 500

```
GET /team-operations/leads?view=all&sort=recently_updated&per_page=20 (employee)
```

```
GET /team-operations/leads?view=all&sort=recently_updated&per_page=20 (admin)
```

Both provided admin and employee accounts receive HTTP 500 from the primary mobile leads list route. While lead detail and update work individually, the main paginated list crashes.

Observed Response / Evidence:


```
{
  "http_status": 500,
  "message": "Server Error"
}
```

Required Fix / Expected Contract:
Resolve the server exception on /team-operations/leads and return the paginated list of leads.

21. Updated Postman collection omits advertised API contracts

```
GET /team-operations/attendance/history
```

```
PATCH /team-operations/team/teams/{id}
```

These routes were advertised in the backend update announcement but are absent from the supplied 141-request collection. Some respond on the live server; their absence is a documentation gap that leads to contract ambiguity.

Request Body / Query:

```
{
  "sms_timing_values": [
    "immediately",
    "scheduled_time",
    "15_minutes_before",
    "30_minutes_before",
    "1_hour_before"
  ],
  "sms_target_values": [
    "client_contact",
    "assigned_representative",
    "both"
  ]
}
```

Required Fix / Expected Contract:
Add complete request/response examples and enums to the Postman collection, including checkout GPS/summary fields.

TASKS & WORK ITEMS AUTHORIZATION

22. Security Flaw: Backend Allows Unauthorized Employees to Update / Complete Other Staff's Tasks (Must Be Blocked with HTTP 403)

```
PATCH /team-operations/work-items/{id} (Authenticated Employee token)
```

While the backend properly auto-assigns new tasks created by an employee to themselves, there is a serious security flaw: the server does NOT verify task ownership on status updates. When tested with live employee credentials (Yeapas), an employee was able to send a PATCH request to a task assigned to a different colleague (Employee ID 1 / NEC Onboarding Test) and change its status to 'completed'. The backend returned HTTP 200 OK ('Work item updated.'), allowing unauthorized employees to close, alter, or complete any colleague's work items.

Request Body / Query:

```
{
  "status": "completed"
}
```

Observed Response / Evidence:

```
{
  "http_status": 200,
  "message": "Work item updated.",
  "work_item": {
    "id": 22,
    "title": "Admin Only Task Not For Yeapas",
    "assignee_employee_id": 1,
    "status": "completed",
    "completed_at": "2026-10-15 14:48:21+00"
  }
}
```

Required Fix / Expected Contract:
CRITICAL SECURITY FIX: Enforce strict backend server-side authorization on PATCH /team-operations/work-items/{id}. Verify that if the authenticated user is not an administrator, work_item.assignee_employee_id MUST match auth()->user()->employee_id. Reject any unauthorized employee attempting to alter another staff's task with HTTP 403 Forbidden ('Unauthorized: You can only update tasks assigned to you.').

23. Security Flaw: Backend Allows Unauthorized Employees to Start Journey, Complete, or Cancel Other Staff's Visits (Must Be Blocked with HTTP 403)

```
POST /team-operations/visits/{id}/start-journey (Authenticated Employee token)
POST /team-operations/visits/{id}/mark-arrived (Authenticated Employee token)
POST /team-operations/visits/{id}/complete (Authenticated Employee token)
POST /team-operations/visits/{id}/reschedule (Authenticated Employee token)
POST /team-operations/visits/{id}/cancel (Authenticated Employee token)
```

CRITICAL FLAW: An employee should NEVER be able to start the journey or submit completion for another employee's visit. However, the backend server currently has ZERO authorization checks on visit lifecycle routes. In testing, an authenticated employee was able to trigger journey start and complete reports on visits assigned to another colleague. The backend must strictly prevent employees from interfering with visits not assigned to them.

Request Body / Query:

```
{
  "visit_id": 14,
  "assigned_to": 1,
  "action": "start-journey",
  "caller_employee_id": 8
}
```

Observed Response / Evidence:

```
{
  "http_status": 200,
  "message": "Visit journey started."
}
```

Required Fix / Expected Contract:

CRITICAL SECURITY FIX: Enforce backend authorization on all visit lifecycle routes: Ensure visit.assignee_employee_id == auth()->user()->employee_id. If an employee is neither the assigned person nor a Super Admin, IMMEDIATELY BLOCK the request with HTTP 403 Forbidden ('Unauthorized: You can only start journey or manage visits assigned to you.').

COMPLETELY MISSING FEATURE CONTRACTS

24. Admin Corporate Expense Management (100% Missing API Endpoints)

```
GET /team-operations/expenses (Admin / Finance)
POST /team-operations/expenses (Admin / Finance)
GET /team-operations/expenses/{id} (Admin / Finance)
PATCH /team-operations/expenses/{id} (Admin / Finance)
DELETE /team-operations/expenses/{id} (Admin / Finance)
GET /team-operations/expenses/summary (Admin / Finance)
```

The mobile application contains a full Corporate Expense Management module (lib/features/expenses/) with live UI for tracking corporate expenditures, filtering by 9 categories (Office & Rent, Utilities & Bills, Software & Tools, Hardware & Devices, Food & Catering, Travel & Conveyance, Marketing & Ads, Contractors & Bonuses, Miscellaneous), recording payment methods (Cash, Company Card, Bank Transfer, bKash, Nagad), attaching receipts, and viewing category breakdown analytics. However, the backend and Postman collection currently have ZERO endpoints for expenses (all requests return HTTP 404). This entire administrative module is forced to run on local demo storage until backend contracts are provided.

Request Body / Query:

```
{
  "title": "Office High-Speed Internet Bill - Oct 2026",
  "amount": 4500.0,
  "category": "utilities",
  "date": "2026-10-15T12:00:00Z",
  "payment_method": "bkash",
  "status": "paid",
  "vendor": "Carnival Internet Ltd",
  "paid_by": "Company Account",
```

```
  "notes": "Monthly dedicated fiber lease line payment",
  "receipt": "<binary file multipart/form-data or storage URL>"
}
```

Observed Response / Evidence:

```
{
  "http_status": 404,
  "message": "Route /team-operations/expenses not found."
}
```

Required Fix / Expected Contract:
Implement the standard 6 REST endpoints under /team-operations/expenses protected by 'team.expenses.manage' permission. Support pagination, query filtering (month, year, category, status, search), receipt file uploads (multipart/form-data), and a monthly summary endpoint (/team-operations/expenses/summary?month=YYYY-MM) returning total spend and category-wise percentage breakdown.

25. System Audit Logs & Security Activity Trails (100% Missing API Endpoints)

```
GET /team-operations/administration/audit-logs (Admin)
```

```
GET /team-operations/administration/audit-logs/{id} (Admin)
```

```
GET /team-operations/administration/audit-logs/export (Admin)
```

The mobile administrative console includes an Audit Log viewer (lib/features/audit/) designed to track sensitive administrative and employee actions (e.g. employee role modifications, salary updates, suspensions/offboarding, lead reassignment, attendance overrides, holiday modifications). The backend provides no audit log endpoints, returning HTTP 404 for /team-operations/administration/audit-logs.

Request Body / Query:

```
{
  "query_parameters": {
    "actor_id": 1,
    "event_type": "employee.suspended",
    "from_date": "2026-10-01",
    "to_date": "2026-10-15",
    "page": 1,
    "per_page": 25
  }
}
```

Observed Response / Evidence:

```
{
  "http_status": 404,
  "message": "Route /team-operations/administration/audit-logs not found."
}
```

Required Fix / Expected Contract:
Implement immutable audit trail endpoints (GET list and GET detail) recording administrative events, IP address, user agent, actor details, and JSON before/after state diffs.

26. Automated SMS Reminders: Missing Background Dispatch Daemon & Outdated Template Specification

```
POST /team-operations/administration/automated-sms/test (Available)
```

```
Background Cron / Queue Worker for Scheduled SMS (MISSING from backend)
```

CRITICAL SMS DELIVERY & TEMPLATE DEFICIENCY:
1. Missing Background Daemon: While manual single-SMS testing exists, the backend has ZERO background queue worker or scheduler that scans pending scheduled follow-ups or visits. When an executive schedules a follow-up with sms_target='client_contact' and sms_timing='15_minutes_before' (or 'scheduled_time'), no SMS is ever sent.

2. Outdated Template In Backend Preview: The backend currently formats automated reminder text with outdated company information and missing meeting links:

OLD / DEFECTIVE TEMPLATE IN CURRENT BACKEND:

Dear Test Client,
This is a reminder from BinGi Hotel Management Technology regarding your scheduled follow-up on 15 Oct 2026 at 4:08 PM.
Best regards,
BinGi Team
For more info visit: https://technology.bingi.com.bd/

REQUIRED OFFICIAL PRODUCTION TEMPLATE (EXACT FORMAT):
Dear [client name],
This is a reminder from BinGi Hotel Management Technology regarding your scheduled follow-up on 15 Oct 2026 at 4:08 PM.
Meeting Link: [Join Meeting]
Best regards,
BinGi Team
For more information: https://technology.bingi.com.bd

Request Body / Query:

```
{
  "scheduled_work_item": {
    "id": 45,
    "type": "follow_up",
    "client_name": "Rahim Khan",
    "client_phone": "01724879284",
    "scheduled_at": "2026-10-15 16:08:00",
    "sms_target": "client_contact",
    "sms_timing": "15_minutes_before",
    "meeting_link": "https://meet.google.com/xyz-qwer-asd"
  }
}
```

Observed Response / Evidence:

```
{
  "status": "no_daemon_executed",
  "error": "No cron/queue worker dispatched. SMS was not received by client contact."
}
```

Required Fix / Expected Contract:
1. BACKGROUND SCHEDULER: Set up a recurring background cron / queue worker running every minute (* * * * * php artisan schedule:run). Scan follow-ups and visits where sms_target IN ('client_contact', 'both') and sms_sent_at IS NULL. Evaluate trigger time against sms_timing (e.g. 15 minutes before scheduled_at, 30 minutes before, or at scheduled_time).

2. EXACT TEMPLATE COMPILATION LOGIC:
• [client name]: Dynamic lead client contact name (if blank, fallback to 'Sir/Madam').
• Scheduled Date & Time: Formatted in local Asia/Dhaka (+06:00) as 'd MMMM yyyy' at 'h:mm a' (e.g. '15 Oct 2026 at 4:08 PM').
• Meeting Link: If work_item.meeting_link is non-empty, render 'Meeting Link: ' + work_item.meeting_link; otherwise render 'Meeting Link: [Join Meeting]'.
• Signature & Website: Must end strictly with:
Best regards,
BinGi Team
For more information: https://technology.bingi.com.bd
(Note: 'For more information:', no trailing slash on URL).

3. DATABASE SUPPORT: Ensure the follow_ups / work_items table has a column 'meeting_link VARCHAR(255) NULL', 'sms_sent_at TIMESTAMP NULL', and 'sms_delivery_status ENUM('pending', 'sent', 'failed') DEFAULT 'pending''.

27. Visit GPS Telemetry & Check-in Verification Coordinates Missing from Detail API

```
POST /team-operations/visits/{id}/start-journey
```

```
POST /team-operations/visits/{id}/mark-arrived
```

```
POST /team-operations/visits/{id}/complete
```

```
GET /team-operations/visits/{id}
```

The mobile client submits live GPS telemetry during visit execution (latitude, longitude, accuracy, departure timestamp, arrival timestamp, and signature data). However, GET /team-operations/visits/{id} returns only basic work-item metadata and completely omits start coordinates, arrival coordinates, travel duration, customer signature, and photo attachments. Management has no way to audit whether an executive actually traveled to the physical client site.

Request Body / Query:

```
{
  "arrival_latitude": 23.7948,
  "arrival_longitude": 90.4043,
  "arrival_accuracy": 8.5,
  "arrived_at": "2026-10-15T15:20:00+06:00"
}
```

Observed Response / Evidence:

```
{
  "returned_visit_detail": {
    "id": 14,
    "title": "Hotel Green View Demo",
    "status": "arrived",
    "missing_fields": [
      "start_coordinates",
      "arrival_coordinates",
      "travel_minutes",
      "signature_url",
      "attachments"
    ]
  }
}
```

Required Fix / Expected Contract:

Persist and return complete field telemetry in GET /team-operations/visits/{id}: 'start_coordinates' ({latitude, longitude, timestamp}), 'arrival_coordinates' ({latitude, longitude, accuracy, timestamp}), 'travel_duration_minutes', 'signature_url', and 'attachments'.

EMPLOYEE ONBOARDING & PROBATION MANAGEMENT

28. Employee Probation Period Tracking, Presets & Agreement PDF Upload (Missing Backend Schema & Contracts)

```
POST /team-operations/employees (Step 2 Onboarding — Multipart/form-data)
PATCH /team-operations/employees/{id} (Employment status & probation updates)
GET /team-operations/employees/{id} (Admin employee profile overview)
GET /team-operations/my-profile (Employee profile overview)
```

NEW REVOLUTIONARY ONBOARDING SPECIFICATION:
The mobile application has implemented an advanced Step 2 onboarding flow for new employees:
1. Employment Type Selector: Radio selection between 'Permanent' and 'Probation Period'.
2. Probation Duration Presets: Instant selection chips for '3 Months', '6 Months', '1 Year', or custom date picker. The UI automatically calculates the exact elapsed days and duration (e.g. 'Duration: 6 Months (180 Days)').
3. Signed Employment Agreement PDF Upload: Dedicated dropzone allowing administrators to upload the official employment contract as a PDF document (max 10MB) during initial employee registration.
4. Dynamic Live Profile Progress Bar: Both the Administrator and the Employee now have dedicated 'Probation Period Progress' cards showing Start Date, End Date, Completed Days, Remaining Days, On-Track Status badge, and an actionable link to View / Download the uploaded Agreement PDF.

BACKEND DEFICIENCY:
The backend 'employees' database table and API endpoints completely lack columns for employment_type, probation dates, and agreement document metadata. When the mobile client submits multipart onboarding data with an agreement PDF, the backend discards the PDF file and saves the employee as an untracked permanent staff.

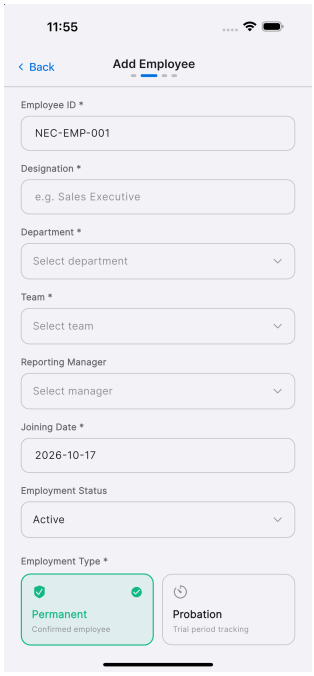


Figure 1: Permanent Employment Selected
Shows standard permanent contract config.

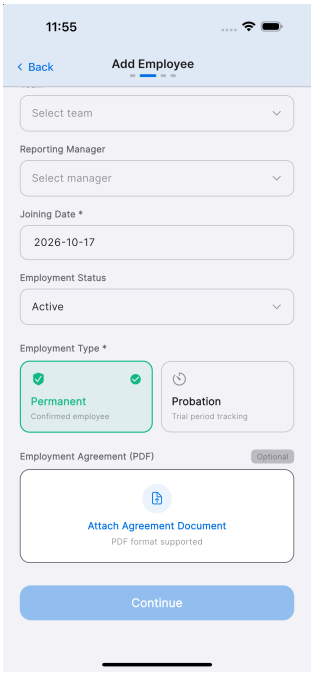


Figure 2: Probation Period & Presets
Presets: 3M, 6M, 1Y with duration calculator.

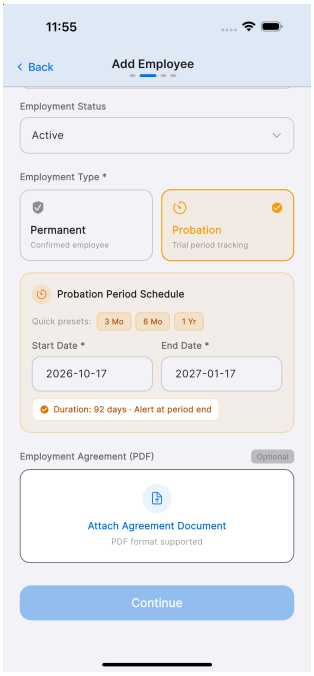


Figure 3: Agreement PDF Upload
Supports official signed contract attachment.

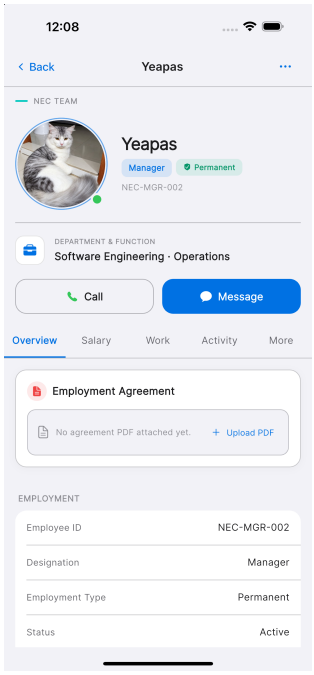


Figure 4: Permanent Profile Header
Verified badge: Permanent Employee.

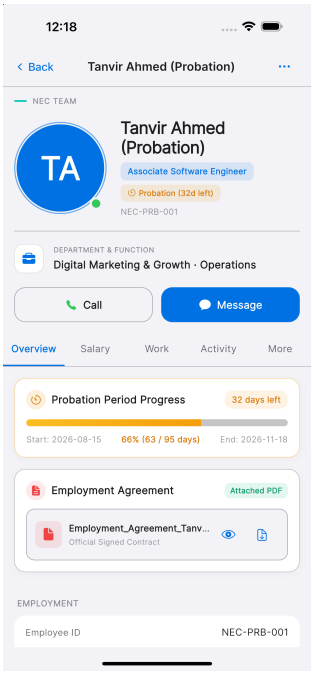


Figure 5: Admin View — Probation Progress
Live countdown (2/180 days) & PDF download.

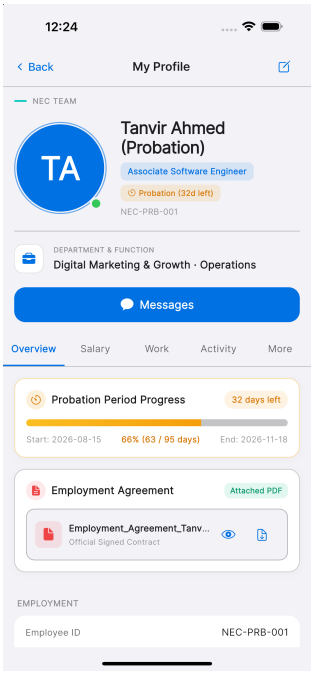


Figure 6: Employee View — My Probation
Employee self-tracking card with progress bar.

Request Body / Query:

```
{
  "name": "Tanvir Ahmed",
  "phone": "01819283746",
  "email": "tanvir@example.com",
  "designation": "Executive - Sales",
  "department_id": 2,
  "employment_type": "probation",
  "probation_start_date": "2026-10-15",
  "probation_end_date": "2027-04-15",
  "agreement_pdf": "<binary application/pdf file: Signed_Agreement_2026.pdf (248 KB)>"
}
```

Observed Response / Evidence:

```
{
  "current_backend_behavior": {
    "saved_employee_id": 15,
    "employment_type": "ignored (defaults to permanent)",
    "probation_start_date": "missing column",
    "probation_end_date": "missing column",
    "agreement_pdf_url": "null (file was not stored)"
  }
}
```

Required Fix / Expected Contract:

1. DATABASE MIGRATION FOR 'employees' TABLE:
ALTER TABLE employees
ADD COLUMN employment_type ENUM('permanent', 'probation') NOT NULL DEFAULT 'permanent' AFTER designation,
ADD COLUMN probation_start_date DATE NULL AFTER employment_type,
ADD COLUMN probation_end_date DATE NULL AFTER probation_start_date,
ADD COLUMN agreement_pdf_url VARCHAR(500) NULL AFTER probation_end_date,
ADD COLUMN agreement_pdf_name VARCHAR(255) NULL AFTER agreement_pdf_url,
ADD COLUMN agreement_pdf_size BIGINT NULL AFTER agreement_pdf_name;

2. MULTIPART FORM-DATA SUPPORT IN POST /team-operations/employees:
Accept 'agreement_pdf' as a file upload (mimes: pdf, max: 10240 KB). Store the document in secure storage (S3 / MinIO / local public storage) and store the accessible HTTPS URL in 'agreement_pdf_url'.

3. PROFILE METRICS SERIALIZATION (GET /employees/{id} & GET /my-profile):
When employment_type == 'probation', the backend must serialize:
• 'employment_type': 'probation'
• 'probation_start_date': '2026-10-15'
• 'probation_end_date': '2027-04-15'
• 'probation_days_total': DATEDIFF(probation_end_date, probation_start_date) -> 180
• 'probation_days_completed': GREATEST(0, DATEDIFF(CURRENT_DATE, probation_start_date)) -> 2
• 'probation_days_remaining': GREATEST(0, DATEDIFF(probation_end_date, CURRENT_DATE)) -> 178
• 'probation_progress_percent': ROUND((probation_days_completed / probation_days_total) * 100, 1) -> 1.1
• 'agreement_pdf_url': 'https://api.bingi.com.bd/storage/agreements/emp_15_agreement.pdf'
• 'agreement_pdf_name': 'Signed_Agreement_2026.pdf'
• 'agreement_pdf_size': 248590

4. AUTOMATED STATUS TRANSITION CRON:
A daily cron job should check for employees whose probation_end_date <= CURRENT_DATE and send an administrative alert: 'Probation period concluded for [Employee Name]. Review for permanent confirmation or extension.'.

CERTIFICATES & TESTIMONIALS LIFECYCLE

29. Employee Certificates & Testimonials Request, Moderation & Delivery (100% Missing Backend API)

```
POST /team-operations/certificates/requests (Employee submit request)
GET /team-operations/certificates/my (Employee view my requests & issued certificates)
GET /team-operations/certificates (Admin view all requests with filter tabs)
POST /team-operations/certificates/{id}/accept (Admin accept request)
POST /team-operations/certificates/{id}/reject (Admin reject request with mandatory reason)
POST /team-operations/certificates/{id}/deliver (Admin fill template & issue official certificate)
GET /team-operations/certificates/{id}/pdf (Download / stream official signed PDF)
```

COMPLETE LIFECYCLE FEATURE CONTRACT MISSING:
When an employee leaves the company or requires verified proof of service (e.g. for higher studies, visa applications, or new career opportunities), they require an official Work Experience Certificate, Testimonial, or Relieving Letter.

The mobile app has implemented the complete end-to-end lifecycle:

1. Employee Request Submission: From the 'More' menu in Profile, employees can request a certificate by selecting Type ('Work Experience Certificate', 'Testimonial', 'Relieving Letter'), specifying Purpose ('Higher Education', 'New Employment', 'Visa / Immigration', 'General'), and providing additional notes.
2. Real-time Status Tracking: The employee can track request states: 'Pending Approval', 'In Preparation', 'Delivered', or 'Rejected'.
3. Admin Management Dashboard: Administrators access 'Certificates & Testimonials' from More > Work Operations with filter chips for All, Pending Approval, In Preparation, Delivered, and Rejected.
4. Moderation Workflow: Admin can Accept or Reject requests. Rejection requires a mandatory reason and sends an immediate alert to the employee.
5. Certificate Template & Issuance: For accepted requests, the Admin fills the official issuance template with Certificate Number (e.g. 'NEC-CERT-2026-0042'), Conduct Assessment ('Exceptional', 'Excellent', 'Good', 'Satisfactory'), Key Contributions, Authorized Signatory Name & Designation (e.g. 'Nazmul Hasan - CEO & Founder'), and Issue Date.
6. Official PDF Generation & Delivery: The system generates an official PDF document with company letterhead, official seal, and signature, sets the status to 'Delivered', and sends an in-app and push notification to the employee.

BACKEND DEFICIENCY:
The backend currently has ZERO endpoints for certificate management (all return HTTP 404). This entire module runs in mock state.

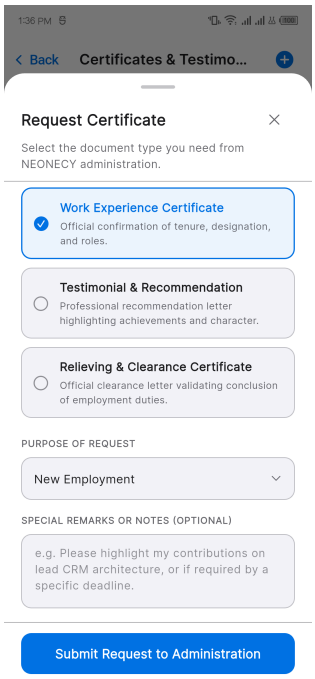


Figure 7: Request Certificate Sheet
Employee chooses Type, Purpose, Notes.

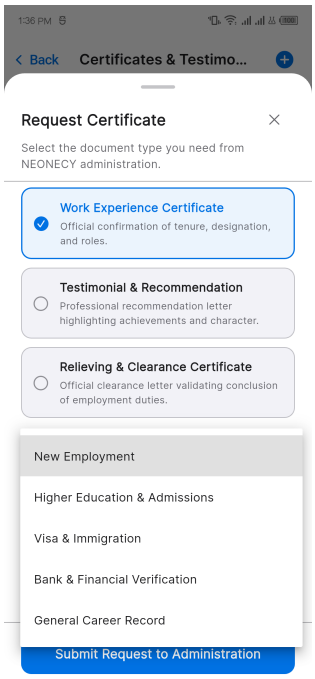


Figure 8: Employee Tracking Card
Real-time status: Under Review.

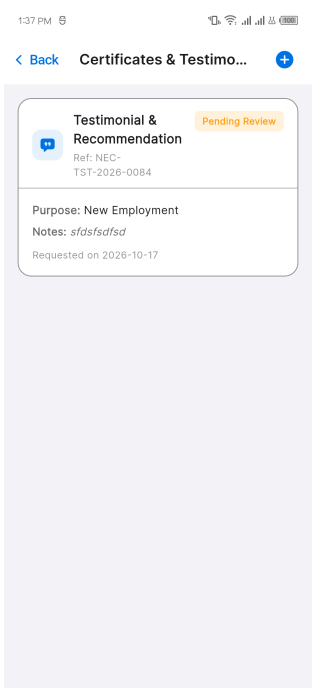


Figure 9: Real-time In-App Notification
Instant confirmation of request submission.

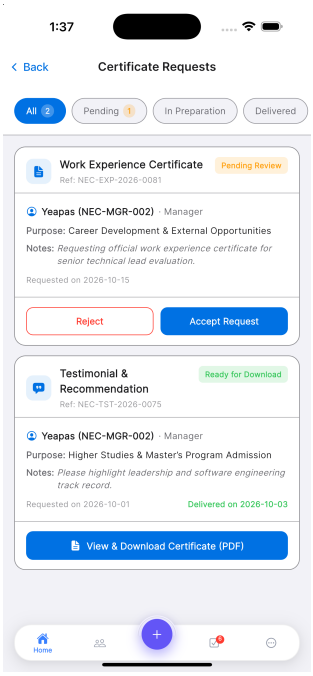


Figure 10: Admin Certificates Console
Filter tabs: All, Pending, In Prep, Delivered.

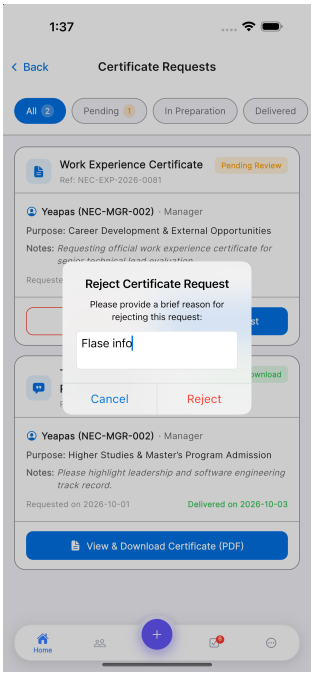


Figure 11: Admin Review Action Sheet
Admin actions: Accept or Reject request.

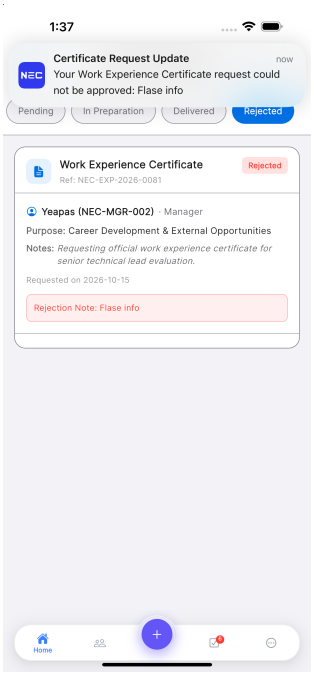


Figure 12: Reject with Reason Modal
Mandatory rejection note alert to employee.

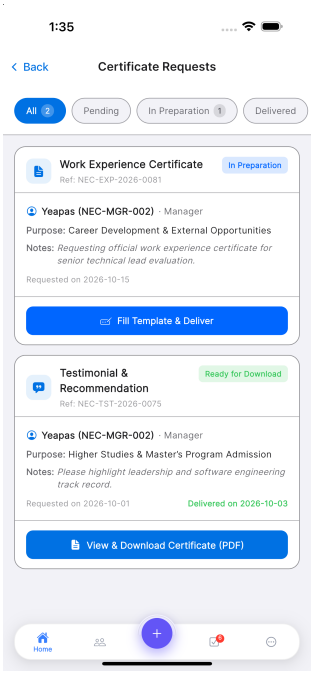


Figure 13: In Preparation Dashboard
Status updated; Fill Template button active.

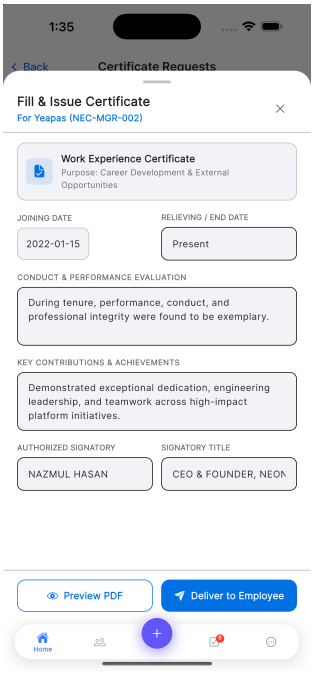


Figure 14: Fill Certificate Template
Input Conduct, Contributions & Signatory.



Figure 15: Official Generated PDF
Branded letterhead, official seal & signature.

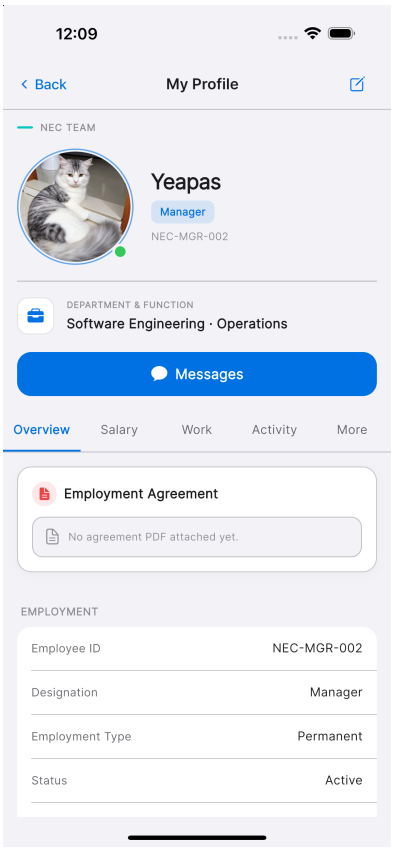


Figure 16: Employee Clean Profile
Role-based access: Admin view cleansed.

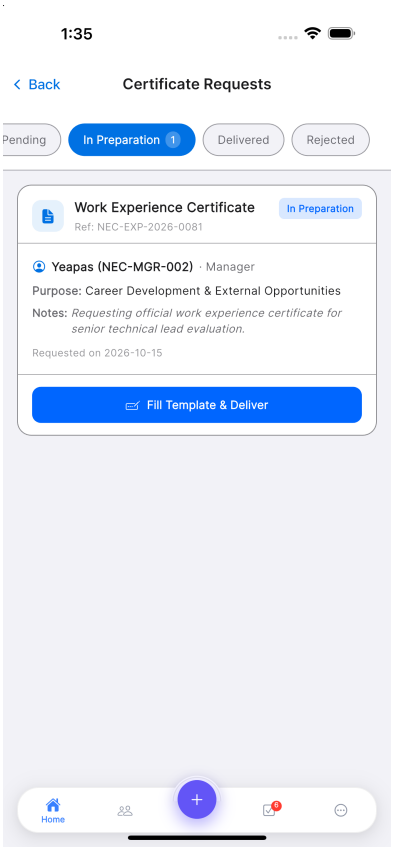


Figure 17: Delivered Request Status
Delivered card with View Issued PDF action.

Request Body / Query:

```
{
  "employee_submit_request": {
    "type": "experience",
    "purpose": "Higher Education",
    "notes": "Applying for Masters abroad, required by university"
  },
  "admin_deliver_payload": {
    "certificate_number": "NEC-CERT-2026-0042",
    "conduct": "exceptional",
    "key_contributions": "Spearheaded mobile CRM development, Flutter offline caching architecture, and
real-time push notification telemetry.",
    "authorized_signatory_name": "Nazmul Hasan",
    "authorized_signatory_designation": "CEO & Founder",
    "issued_at": "2026-10-17T12:00:00Z"
  }
}
```

Observed Response / Evidence:

```
{
  "http_status": 404,
  "message": "Route /team-operations/certificates not found."
}
```

Required Fix / Expected Contract:

1. DATABASE TABLE: 'certificate_requests'
CREATE TABLE certificate_requests (
id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
certificate_number VARCHAR(64) UNIQUE NULL,
employee_id BIGINT UNSIGNED NOT NULL,
requested_by_user_id BIGINT UNSIGNED NOT NULL,
type ENUM('experience', 'testimonial', 'relieving') NOT NULL,
purpose VARCHAR(255) NOT NULL,
notes TEXT NULL,
status ENUM('pending', 'in_preparation', 'delivered', 'rejected') NOT NULL DEFAULT 'pending',

```
rejection_reason TEXT NULL,
conduct ENUM('exceptional', 'excellent', 'good', 'satisfactory') NULL,
key_contributions TEXT NULL,
authorized_signatory_name VARCHAR(150) NULL,
authorized_signatory_designation VARCHAR(150) NULL,
issued_at TIMESTAMP NULL,
pdf_url VARCHAR(500) NULL,
created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
FOREIGN KEY (employee_id) REFERENCES employees(id) ON DELETE CASCADE,
FOREIGN KEY (requested_by_user_id) REFERENCES users(id) ON DELETE CASCADE
);
```

2. REST ENDPOINT CONTRACTS:

• POST /team-operations/certificates/requests: Accepts type, purpose, notes. Creates row with status='pending'. Dispatches notification to Admins: 'New Certificate Request from [Employee Name]'.

• GET /team-operations/certificates/my: Returns authenticated employee's list of certificate requests and issued documents.

• GET /team-operations/certificates: Admin paginated list with query parameters: ?status=pending|in_preparation|delivered|rejected&type=experience|testimonial|relieving&search=...&page=1&per_page=20.

• POST /team-operations/certificates/{id}/accept: Updates status to 'in_preparation'. Dispatches notification to employee: 'Your certificate request has been accepted and is being prepared.'.

• POST /team-operations/certificates/{id}/reject: Accepts { 'reason': string }. Updates status to 'rejected' and stores rejection_reason. Dispatches alert to employee with the reason.

• POST /team-operations/certificates/{id}/deliver: Accepts template parameters (certificate_number, conduct, key_contributions, authorized_signatory_name, authorized_signatory_designation, issued_at). Compiles the official PDF, stores it, sets status='delivered' and pdf_url. Dispatches notification to employee: 'Your official certificate has been delivered and is ready for download.'.

• GET /team-operations/certificates/{id}/pdf: Returns the generated PDF stream with Content-Type: application/pdf.